

SALES SAVVY

E-Commerce Management System

Software Requirements Specification (SRS)

Version: 1.0

Project: Sales Savvy

Application Type: Full-Stack E-Commerce Web Application

1. Introduction

1.1 Project Overview

Sales Savvy is a full-stack e-commerce application designed for small and medium-sized businesses.

The system provides two separate interfaces:

- **Admin Interface** – used to manage users, products, categories, orders, payments, and sales reports.
- **Customer Interface** – used by customers to register, browse products, manage their shopping cart, place orders, make payments, and track orders.

The application will be developed using React.js for the frontend and Spring Boot for the backend, with MySQL as the database.

The system will use Spring Security and JWT-based authentication for securing APIs and BCrypt password hashing for securely storing user passwords.

Razorpay will initially be used as the payment gateway.

2. Objectives

The main objectives of Sales Savvy are:

1. Provide businesses with a simple online store management system.
2. Allow administrators to manage products, categories, users, orders, and payments.
3. Allow customers to browse and search products easily.
4. Provide customers with shopping cart and checkout functionality.
5. Allow customers to securely make online payments.
6. Allow customers to track their orders.
7. Provide administrators with sales and order-related reports.
8. Build the application with a scalable architecture so that new features can be added later.
9. Provide secure authentication and authorization.

10. Dockerize the application so it can be deployed easily.

3. Target Users

The system will have two primary types of users.

3.1 Administrator

Administrators can:

- Log in through the administrator login system.
- Access the admin dashboard.
- Manage customers and other users.
- Create, update, deactivate, and delete user accounts.
- Assign roles to users.
- Create, update, and delete products.
- Manage product images.
- Create, update, and delete categories.
- Manage product tags.
- View and manage customer orders.
- Approve or cancel orders.
- Update order status.
- Manage shipping-related information.
- View payment information.
- Manage payment status.
- Generate sales and performance reports.

3.2 Customer

Customers can:

- Register for an account.
- Verify their email address.
- Log in through the customer login system.
- Manage their personal information.
- Browse products.
- Search products.
- Filter products by category and tags.
- View product details, price, and availability.
- Add products to their shopping cart.
- Remove products from their cart.
- Update product quantities.
- View total cart cost.
- Select shipping options.
- Place orders.
- Make online payments.
- View order history.
- Track order status.

- View delivery status.
-

4. Functional Requirements

FR-01: Separate Authentication Systems

The system shall provide separate login interfaces for:

- Administrators
- Customers

Administrators shall have access to the administrator dashboard only after successful authentication and authorization.

Customers shall have access to customer-specific functionality after successful authentication.

FR-02: Customer Registration

Customers shall be able to create an account using their required personal information.

The registration process shall include:

- Email
- Password
- Required personal information

The system shall verify the customer's email address before allowing full account access.

Passwords shall never be stored as plain text.

Passwords shall be securely hashed using `BCryptPasswordEncoder`.

FR-03: Authentication and Authorization

The application shall use:

- Spring Security
- JWT-based authentication
- BCrypt password hashing
- Role-based authorization

The system shall support at least the following roles:

- `ADMIN`
- `CUSTOMER`

The backend shall restrict protected resources based on the authenticated user's role.

For example:

- Admin APIs shall only be accessible to administrators.
 - Customer-specific APIs shall require customer authentication.
 - Public product browsing APIs may be accessible without authentication where appropriate.
-

5. User Management

Administrators shall be able to:

- View users.
- Create users.
- Update users.
- Deactivate users.
- Delete users where appropriate.
- Assign user roles.
- View basic user information.

Customers shall be able to:

- View their own profile.
- Update their personal information.
- Manage their account details.

A customer shall not be able to modify another customer's account.

6. Product Catalog Management

6.1 Product Management

Administrators shall be able to:

- Add products.
- Edit products.
- Delete products.
- Update product prices.
- Update product availability.
- Manage product images.
- Assign products to categories.
- Assign tags to products.

A product may contain information such as:

- Product name
- Description

- Price
- Stock quantity
- Availability
- Product images
- Category
- Tags
- Created date
- Updated date

The exact fields will be finalized during database design.

6.2 Category Management

Administrators shall be able to:

- Create categories.
- Update categories.
- Delete categories.
- View categories.

Products shall be associated with appropriate categories.

6.3 Tag Management

Products shall support tags.

Customers shall be able to use tags as one of the ways to discover products.

7. Customer Product Browsing

Customers shall be able to:

- View available products.
- View product details.
- View product price.
- View product availability.
- Search products by name.
- Search/filter products by category.
- Search/filter products by tags.

The product browsing interface shall provide a user-friendly experience.

8. Shopping Cart

Authenticated customers shall have a shopping cart.

Customers shall be able to:

- Add products to the cart.
- Remove products from the cart.
- Increase product quantity.
- Decrease product quantity.
- View product details in the cart.
- View product prices.
- View selected quantities.
- View the total cart cost.

The system shall validate product availability and stock before allowing an order to be placed.

9. Order Management

9.1 Customer Order Creation

Customers shall be able to:

- Review their cart.
- Select a shipping option.
- Review order details.
- Place an order.
- Proceed to payment.

An order shall contain relevant information such as:

- Customer
 - Ordered products
 - Quantities
 - Prices
 - Shipping information
 - Shipping option
 - Total amount
 - Order status
 - Payment status
 - Order date
-

9.2 Order Status

The application shall support order statuses including:

1. PENDING
2. APPROVED
3. SHIPPED
4. DELIVERED
5. CANCELLED

The exact order-state transition rules will be defined during system design.

9.3 Customer Order Tracking

Customers shall be able to:

- View current orders.
- View previous orders.
- View order details.
- View order status.
- Track delivery status.
- View purchase history.

Customers shall only be able to access their own orders.

9.4 Administrator Order Management

Administrators shall be able to:

- View orders.
 - View order details.
 - Approve orders.
 - Cancel orders.
 - Update order status.
 - Mark orders as shipped.
 - Mark orders as delivered.
 - View customer and order information.
-

10. Payment Gateway Integration

The application shall integrate with **Razorpay** as the initial payment gateway.

Customers shall be able to:

- Initiate payment during checkout.

- Complete payment through Razorpay.
- Receive payment confirmation.

The backend shall verify payment results before treating an order as successfully paid.

The system shall maintain payment information such as:

- Payment identifier
- Order identifier
- Amount
- Payment status
- Payment timestamp
- Payment gateway information

Administrators shall be able to view payment information and payment status.

Sensitive payment credentials and API secrets shall not be stored in source code or committed to GitHub.

11. Reports

Administrators shall be able to generate basic sales and performance reports.

Reports may include:

- Total sales
- Number of orders
- Completed orders
- Cancelled orders
- Sales over a selected period
- Best-selling products

The exact reports and filters will be finalized during implementation.

12. Security Requirements

The application shall implement appropriate security measures.

12.1 Authentication

The application shall use:

- Spring Security
- JWT authentication
- Secure password hashing

Passwords shall be hashed using:

BCryptPasswordEncoder

Plain-text passwords shall never be stored in the database.

12.2 Authorization

The application shall implement role-based access control.

Example:

```
ADMIN
  ↓
Admin Dashboard
  ↓
User / Product / Order / Payment Management

CUSTOMER
  ↓
Customer Interface
  ↓
Browse / Cart / Checkout / Orders
```

Users shall not be allowed to access resources outside their permissions.

12.3 Data Transmission

Production communication shall use HTTPS to protect data transmitted between clients and the server.

12.4 Sensitive Information

The following shall not be hard-coded into the source code:

- Database passwords
- JWT secrets
- Razorpay API secrets
- Other private credentials

Sensitive configuration shall be supplied through environment variables or appropriate deployment configuration.

13. Non-Functional Requirements

13.1 Performance

The application should:

- Support approximately 100 simultaneous customers for the initial target.
- Aim for API response times below 2 seconds under normal expected load.
- Avoid unnecessary database queries.
- Use appropriate database indexes and query optimization where required.

Performance requirements may be refined after testing.

13.2 Scalability

The application shall be designed so that additional functionality can be added later without major architectural changes.

The system should allow future additions such as:

- Additional payment gateways
 - Additional reporting functionality
 - Additional product features
 - Additional user roles
 - Additional business functionality
-

13.3 Usability

The application should:

- Have a clear user interface.
 - Provide simple navigation.
 - Display meaningful validation messages.
 - Provide understandable error messages.
 - Work well on common desktop and mobile screen sizes.
-

13.4 Maintainability

The backend shall follow a layered architecture.

The initial architecture shall follow:

React Frontend

↓

Spring Security



Controller



Service



Repository



MySQL

The application should use clear separation of responsibilities.

14. Technology Stack

Frontend

- React.js
- JavaScript
- HTML
- CSS
- Bootstrap

Backend

- Java
- Spring Boot
- Spring Web
- Spring Security
- Spring Data JPA

Database

- MySQL

Authentication and Security

- Spring Security
- JWT
- BCryptPasswordEncoder

Payment

- Razorpay

Version Control

- Git

- GitHub

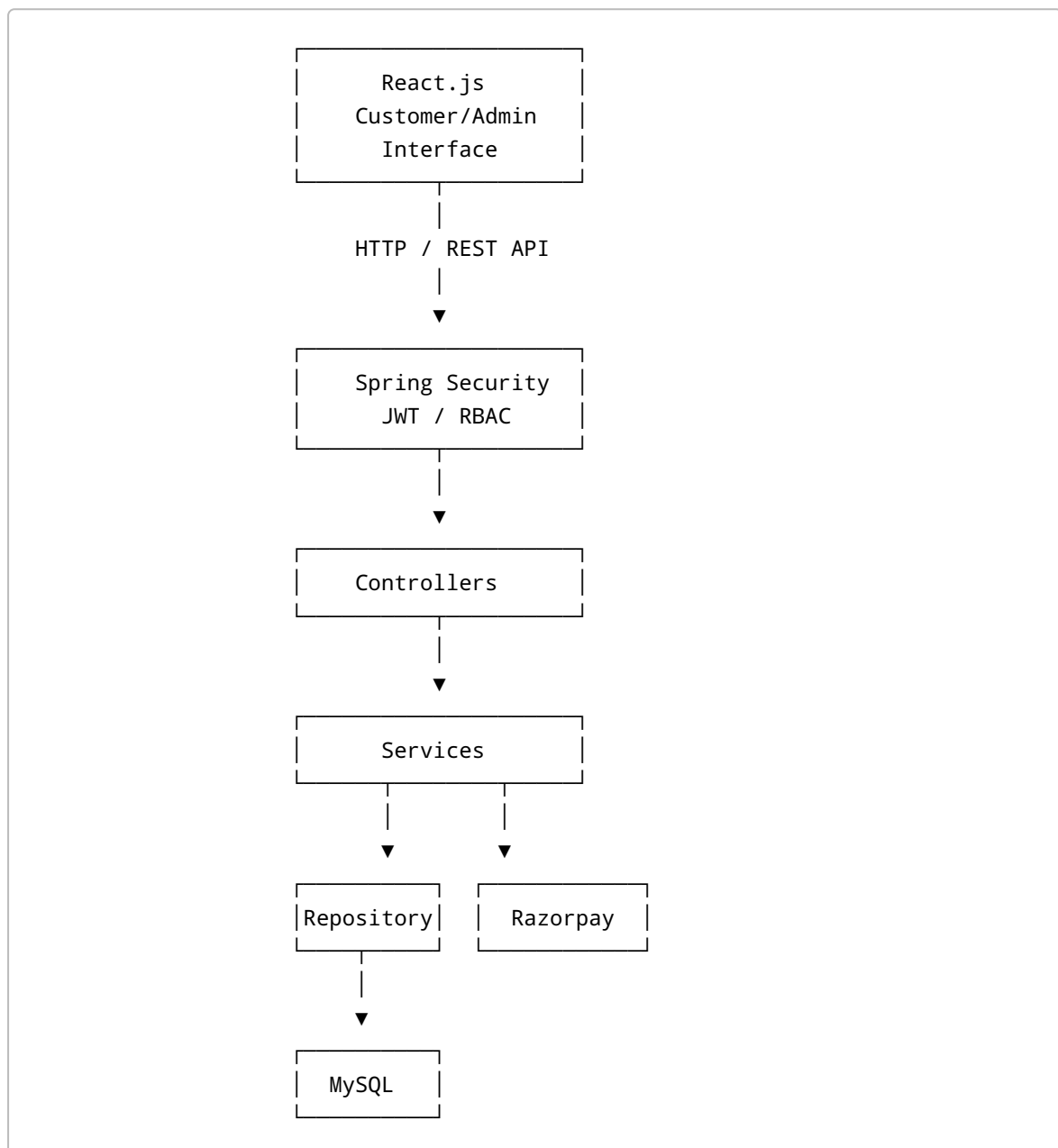
DevOps

- Docker
- Docker Compose

15. System Architecture

The application will follow a client-server architecture.

High-level architecture:



16. Scope

16.1 Included in the Project

The first version shall include:

- Admin authentication
- Customer registration
- Customer email verification
- Customer authentication
- JWT authentication
- Role-based authorization
- User management
- Product management
- Category management
- Product tags
- Product browsing
- Product searching
- Shopping cart
- Checkout
- Shipping option selection
- Order management
- Order tracking
- Payment integration
- Payment status management
- Basic sales reports
- React frontend
- Spring Boot REST API
- MySQL database
- Docker configuration
- Docker Compose configuration
- Project documentation

17. Scope Limitations

The first version shall NOT include:

- Product recommendation systems
- AI-based recommendations
- Multiple languages
- Multiple currencies
- Multiple payment gateways simultaneously
- Advanced business intelligence
- Complex warehouse management

The initial application will use:

- One language
- One currency
- Razorpay as the initial payment provider

Additional functionality may be added in future versions.

18. Project Milestones

Milestone 1 — Requirements

- Finalize requirements
- Define project scope
- Define functional requirements
- Define non-functional requirements

Deliverable: Requirements Specification

Milestone 2 — System Design

- Define application architecture
- Identify system modules
- Identify entities
- Design database relationships
- Define API structure
- Decide authentication architecture

Deliverable: System Design + ER Diagram

Milestone 3 — Project Setup

- Create Git repository
- Create Spring Boot backend
- Create React frontend
- Configure MySQL
- Configure development environment
- Configure `.gitignore`
- Make initial Git commit

Deliverable: Working project skeleton

Milestone 4 — Backend Foundation

- Configure Spring Boot

- Configure MySQL
- Configure JPA
- Establish project package structure
- Create initial entities
- Create repositories
- Create basic services and controllers

Deliverable: Working backend foundation

Milestone 5 — Authentication and Security

- Customer registration
- Password hashing with BCrypt
- Email verification
- Customer login
- Admin login
- JWT generation
- JWT validation
- Spring Security configuration
- Role-based authorization

Deliverable: Secured authentication system

Milestone 6 — Product Catalog

- Category management
- Product management
- Product tags
- Product images
- Product browsing
- Product search
- Product availability

Deliverable: Working product catalog

Milestone 7 — Shopping Cart

- Add product
- Remove product
- Update quantity
- Calculate totals
- Validate stock

Deliverable: Working shopping cart

Milestone 8 — Orders

- Checkout
- Shipping options
- Create orders
- Order items
- Order history
- Order tracking
- Admin order management
- Order status management

Deliverable: Working order system

Milestone 9 — Payment

- Razorpay integration
- Payment initiation
- Payment verification
- Payment status
- Payment information
- Order/payment consistency

Deliverable: Working payment system

Milestone 10 — React Frontend

- Customer interface
- Admin interface
- Login/register screens
- Product pages
- Cart
- Checkout
- Order pages
- Admin dashboard
- User management
- Product management
- Order management

Deliverable: Complete frontend

Milestone 11 — Integration

- Connect React with Spring Boot APIs
- Handle authentication
- Handle JWT
- Handle API errors
- Handle loading states

- Complete customer workflows
- Complete admin workflows

Deliverable: Fully integrated application

Milestone 12 — Testing

Test:

- Authentication
- Authorization
- User management
- Product management
- Cart
- Orders
- Payments
- Admin functionality
- Customer functionality
- API validation
- Error handling
- Performance

Deliverable: Tested application

Milestone 13 — Dockerization

Create:

- Backend Docker configuration
- Frontend Docker configuration
- MySQL container configuration
- Docker Compose configuration

Verify that the complete application can run using Docker Compose.

Deliverable: Dockerized application

Milestone 14 — Deployment

Prepare the application for deployment.

Tasks include:

- Production configuration
- Environment variables
- HTTPS
- Database configuration

- Docker deployment
- Application verification

Deliverable: Deployment-ready application

Milestone 15 — Documentation

Documentation shall include:

- Project overview
- Features
- Technology stack
- Architecture
- Database design
- API documentation
- Setup instructions
- Environment configuration
- Docker instructions
- Deployment instructions
- Git/GitHub workflow
- Known limitations

Final Deliverable: Fully functional, documented, and Dockerized Sales Savvy e-commerce system.

19. Development Approach

The project shall be developed incrementally.

Each feature will follow this general process:

```
Requirement
  ↓
Design
  ↓
Implementation
  ↓
Testing
  ↓
Git Commit
  ↓
Next Feature
```

The project will use Git from the beginning rather than uploading the completed project to GitHub only after production.

Meaningful changes shall be committed regularly with descriptive commit messages.

Example:

```
Initial project setup
Configure MySQL connection
Add User entity
Add user repository
Implement customer registration
Add BCrypt password hashing
Implement JWT authentication
Add product management
```

Secrets and sensitive credentials shall never be committed to the repository.

20. Future Enhancements

Possible future versions may include:

- Multiple payment gateways
- Multiple currencies
- Multiple languages
- Product recommendation system
- Advanced analytics
- Inventory management
- Discount and coupon system
- Product reviews and ratings
- Wishlist
- Notifications
- Advanced shipping management

These features are outside the scope of version 1.0.

21. Definition of Done

Sales Savvy version 1.0 will be considered complete when:

- Customers can register and authenticate.
- Email verification works.
- Passwords are securely hashed using BCrypt.
- JWT authentication works.
- Role-based access control works.
- Administrators can manage users.
- Administrators can manage products and categories.
- Customers can browse and search products.
- Customers can manage their cart.
- Customers can place orders.
- Customers can make payments through Razorpay.

- Payment results are verified by the backend.
 - Customers can view order history and status.
 - Administrators can manage orders.
 - Basic sales reports are available.
 - React and Spring Boot are integrated.
 - Application security requirements are implemented.
 - Application is tested.
 - Application is Dockerized.
 - Docker Compose can start the application.
 - Project documentation is complete.
 - Application is ready for deployment.
-

22. Project Development Rule

This project will be developed as a learning project.

Implementation will be performed incrementally rather than copying a complete application.

For each feature:

1. Understand the requirement.
2. Understand the architecture.
3. Design the data.
4. Implement the backend.
5. Test the backend.
6. Implement the frontend.
7. Connect frontend and backend.
8. Test the complete feature.
9. Commit the work using Git.
10. Move to the next feature.

The goal is not only to produce a working e-commerce application but also to understand how a real-world Spring Boot + React application is designed, developed, tested, version-controlled, and deployed.